Linde
GENIE

Spécifications du projet:.....4
Répartition des tâches :.....6
PHASE1 : Étude des Capteurs de Pression Atmosphérique et d'Anémomètres :.....7
Capteur de pression atmosphérique:.....8
Comparaison des Capteurs : BMP180 vs BMP280 :.....9
Étude du BMP280 :.....11
Picoscope :.....24
Conclusion du BMP280 :.....27
Capteur anémomètre:.....27
Programmation:.....35
Phase 2 : Concevoir une carte PCB avec capteur C3-BMP280 et développer un programme d'alerte en cas de risque de grêle (niveau 2 ou 3).....40
Carte PCB :.....40

Introduction :

Je vais vous présenter mon rapport de projet individuel. Je m’appelle Adam Boumaad, étudiant en deuxième année de BTS CIEL B au Campus Ozanam. Durant cette année, j’ai participé à un projet collectif portant sur l’étude et l’amélioration d’une station météo.

Le projet s’inscrit dans le cadre de l’installation d’un système d’alerte sur un vignoble. L’objectif est de reproduire et optimiser une solution existante en intégrant de nouvelles fonctionnalités, notamment en diversifiant les moyens de transmission des données météorologiques et des alertes.

Le principe consiste à limiter le risque de grêle en favorisant la formation de pluie. Dans un nuage susceptible de générer de la grêle, les noyaux de condensation de grande taille accélèrent la formation des gouttes de pluie, qui, en montant en altitude, se transforment en grêlons. Ces noyaux de condensation, souvent faits de petites impuretés, aident l'humidité dans l'air à se transformer en gouttes plus grosses. À cause de leur poids, ces gouttes se transforment en grêle lorsqu'elles montent dans les zones plus froides du nuage et gèlent.

Pour limiter ce phénomène, on utilise des sels hygroscopiques, qui attirent l'humidité de l'air et favorisent la formation de gouttes d'eau plus petites. Cela empêche la création de gros noyaux de condensation, réduisant ainsi la formation de grêle. En effet, en diminuant la taille des noyaux, les sels hygroscopiques évitent l'accumulation rapide d'humidité et la croissance des gouttes, ce qui permet de prévenir la formation de grêlons.

Lors d'une alerte, des ballons biodégradables gonflés à l’hélium, équipés de torches libérant ces sels, sont envoyés dans les zones orageuses à risque. Bien que cette méthode soit efficace, elle est coûteuse et n’est utilisée que lorsque le risque de grêle est élevé. La décision d'agir est généralement prise par le chef de culture, en concertation avec son équipe, après avoir analysé les données météorologiques.

À l’aide d’une plateforme numérique dotée d'un outil d'évaluation des risques de grêle permet d'identifier les nuages menaçant. Les viticulteurs, informés par SMS, peuvent suivre en temps réel le déplacement des cellules orageuses, avec trois niveaux d'alerte : faible, modéré et fort, pour signaler l'intensité du danger.

Spécifications du projet:

Dans cette partie, je vais expliquer les spécificités du projet, en le divisant en trois sections principales. Tout d'abord, la section **"Ce que le client veut"** détaillera les besoins du client, notamment la collecte de données météorologiques fiables, l'envoi d'alertes SMS avec différents niveaux de risque, ainsi que l'accès à un site web pour consulter ces données en temps réel. Ensuite, la section **"Aspects techniques"** abordera les solutions technologiques mises en place pour répondre à ces besoins, telles que les outils de collecte des données (station météo, GPS), le traitement des informations et la gestion des alertes. Enfin, la section **"Station-Vigne et Serveur-Manoir"** expliquera le rôle spécifique de la Station-Vigne, qui collecte les données météorologiques sur le terrain, et du Serveur-Manoir, qui centralise ces informations, permettant ainsi de gérer et de déclencher à distance les actions nécessaires, comme le lâcher de ballons anti-grêle. Ces trois parties permettent de comprendre les exigences du projet et la manière dont elles sont mises en œuvre techniquement.

En effet, je vais décrire les besoins et attentes du client. Le client désire avoir accès à des données météorologiques fiables provenant de chaque stationvigne installée sur le domaine. Il souhaite également recevoir des alertes par SMS concernant le risque de grêle, ces alertes étant classées en trois niveaux : faible, modéré et élevé. Le client doit pouvoir consulter l'ensemble des données collectées via un site web hébergé sur le serveur du domaine, appelé Serveur-Manoir. Il souhaite également avoir la possibilité de gérer ce serveur et d'accéder à la base de données qui enregistre quotidiennement les données météorologiques à des fins d'archivage. Enfin, le client veut pouvoir déclencher à distance le lâcher de ballons anti-grêle en cas de nécessité.

De plus, pour les aspects techniques le système a pour objectif de collecter les données météorologiques nécessaires à l'évaluation du risque de grêle, telles que la température, l'humidité relative, la pression atmosphérique et la vitesse du vent, depuis la Station-Vigne. Il doit également recueillir les informations de géolocalisation des données via GNSS et horodater ces informations. La Station-Vigne analyse ces données et évalue le risque de grêle. Le niveau de ce risque est ensuite transmis au Serveur-Manoir, situé sur le site de production. Une alerte SMS est envoyée aux responsables identifiés, tels que le chef de culture et le responsable informatique. Le système permet également une connexion via smartphone au site web du vignoble pour consulter les données météorologiques et toutes les informations pertinentes. Le risque de grêle peut être validé en croisant les

données collectées avec celles provenant d'une base de données externe, comme Météo-France
(optionnel). Le lâcher du ballon anti-grêle peut être déclenché à distance, soit depuis un smartphone, soit depuis le poste central du vignoble. Enfin, les données météorologiques collectées sont transférées quotidiennement au serveur pour l’archivage et les statistiques. Le système doit également intégrer une gestion de l’autonomie énergétique de la Station-Vigne, en utilisant des solutions comme des panneaux solaires et des batteries, afin de garantir un fonctionnement continu même en l'absence de connexion au réseau électrique.

Enfin, pour les deux entités secondaires qui sont la station-vigne et le serveur manoir. Au niveau de la Station-Vigne, il est essentiel de mesurer les principales données météorologiques telles que la température, l'humidité, la pression atmosphérique et la vitesse du vent. Une plateforme de traitement des données, à faible consommation d'énergie comme Arduino ou une alternative similaire, doit être mise en place. Des nœuds LoRa doivent être installés et connectés à une passerelle LoRa située au Serveur-Manoir. Un module GPS doit également être intégré pour fournir des données de localisation, et un service d’envoi d’alertes par SMS doit être activé. En outre, la station doit être équipée d’un système garantissant son autonomie énergétique, par exemple avec des panneaux solaires ou des batteries.

Du côté du Serveur-Manoir, il est nécessaire de déployer une infrastructure réseau complète, incluant un serveur principal, une passerelle LoRa et un PC dédié à l’administration. Le serveur sera chargé de collecter, analyser et archiver les informations transmises par la Station-Vigne. Enfin, un site web devra être conçu pour permettre la visualisation des données recueillies, qu'elles soient en temps réel, quotidiennes ou sauvegardées.

Répartition des tâches :

Étudiant 1 : Benoît Borek

Phase 1 : Étude capteur de température et Humidité.

Phase 2 : GSM,Lora,lier GNSS et anémomètre.

Phase 3 : Inscription serveur TTN,passerelle Lora,afficher données(tableau,graphique).

Étudiant 2 : Boumaad Adam

Phase 1 : Étude capteur de pression atmosphérique,anémomètre.

Phase 2 : Associer capteur de température,Humidité et pression atmosphérique.

Phase 3 : Site web(page d'accueil,page archivage,page déclenchement).

Étudiant 3 : Alban

Phase 1 : Étude GPS,Horodatation.

Phase 2 : Batterie,panneau solaire,Régulateur.

Phase 3 : 433 Mhz,Réalisation opération de maintenance et/ou de réparation tester la fonction déclenchement.

PHASE1 : Étude des Capteurs de Pression Atmosphérique et d'Anémomètres :

Dans le cadre de la première phase du projet, en tant qu'étudiant numéro 2, ma tâche principale consiste à réaliser une étude approfondie sur les capteurs de pression atmosphérique et les anémomètres.

Cette phase vise à comprendre le fonctionnement de ces capteurs essentiels dans la mesure des variables environnementales. Je vais commencer par analyser les différents types de capteurs de pression atmosphérique, en examinant leurs caractéristiques techniques telles que leur précision, leur plage de mesure et leurs applications dans des domaines comme la météorologie et les systèmes de contrôle de l'environnement.

De même, l'étude des anémomètres, utilisés pour mesurer la vitesse du vent, permettra de comprendre leur fonctionnement, leurs avantages, et leur fiabilité dans des conditions variables.

À travers cette étude, mon objectif est de sélectionner les capteurs les plus adaptés pour les étapes suivantes du projet, en mettant l'accent sur la fiabilité et la précision des données qu'ils génèrent.

Capteur de pression atmosphérique:

L’utilisation du capteur de pression atmosphérique est essentielle pour détecter la tombée de grêle. Une baisse rapide de la pression peut signaler l’approche d’une perturbation, comme un orage ou une cellule grêleuse, car elle est liée à la formation de nuages cumulonimbus, responsables de la grêle. Combiné à d’autres capteurs, comme ceux mesurant la température, l’humidité et la vitesse du vent, un capteur de pression améliore l’évaluation des risques de grêle.

Des seuils de pression associés à la vitesse du vent peuvent déclencher des alertes SMS pour avertir les viticulteurs en temps réel. Le capteur BMP280, moderne et précis, est idéal pour une station météorologique autonome grâce à son faible coût et sa faible consommation d’énergie.

Avant de choisir le BMP280, j'ai comparé ses spécifications avec celles du BMP180, un modèle plus ancien, pour m'assurer de la meilleure option.

Lien de la datasheet : https://cdn-shop.adafruit.com/datasheets/BST-BMP280DS001-11.pdf

Comparaison des Capteurs : BMP180 vs BMP280 :

Paramètres	BMP180	BMP280	Avantage par rapport au projet
Dimensions	3.6 × 3.8 mm	2.0 × 2.5 mm	Le BMP280 est plus compact, facilitant l'intégration.
Consommation de courant	12 µA	2.7 µA	Le BMP280 est 4 fois plus économe en énergie, idéal pour une station autonome.
Résolution de pression	1 Pa	0,16 Pa	Le BMP280 permet des mesures de pression plus précises, cruciales pour évaluer le risque de grêle.
Options de filtrage	Aucune	Cinq bandes passantes	Le BMP280 réduit le bruit dans les mesures, augmentant la fiabilité des données.

Tableau pour la consommation d'énergie :

BMP 280	P(W)=9.042*10^{-6}	U(V)=3.3	I(A)=2.74*10^{-6}
BMP 180	P(W)=3.96 × 10^{-5}	U(V)=3.3	I(A)=12*10^{-6}

Finalement, le BMP180 offre des caractéristiques intéressantes, mais son moindre résolution de pression (1 Pa contre 0,16 Pa pour le BMP280) et son plus faible éventail de fonctionnalités, telles qu'une plage de fonctionnement plus étroite, l'ont fait pencher en faveur du BMP280. Ce dernier présente une meilleure précision, une meilleure résolution de température et des options de filtrage plus avancées. En outre, le BMP280 consomme moins d'énergie (2,7 µA contre 12 µA pour le BMP180), ce qui est un critère essentiel pour un projet autonome.
De plus, le BMP280 propose des interfaces I²C et SPI, offrant une plus grande flexibilité et une meilleure compatibilité avec les microcontrôleurs, ce qui est crucial pour les projets nécessitant différents modes de communication. Pour ces raisons, j'ai choisi le BMP280 pour mon projet de station météorologique.

Le BMP280 est alimenté par 3,3 V, prend en charge le suréchantillonnage et offre des interfaces I²C et SPI, facilitant son intégration dans divers projets avec différents microcontrôleurs.

Étude matérielle :

Le module BMP280 est conçu de la manière suivante : au centre se trouve le capteur, qui possède un petit trou permettant de mesurer la pression atmosphérique. Autour de ce capteur, on trouve 4 résistances et 2 condensateurs. Le module comporte 6 broches : VCC, GND, SCL, SDA, CSB et SDO.

Pins	Description
VIN	Il sert d'alimentation d'entrée. (+3,3V).
GND	La masse du module.
SCL	Entrée pour l'horloge.
SDA	Entrée/Sortie pour les données.
CSB	Chip Select (SPI)
SDO	Data out

Étude du BMP280 :

Maintenant je dois comprendre le fonctionnement du capteur lui-même.
En utilisant la documentation technique que j'avais trouvé auparavant. Lien : https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf

1) Le BMP280 est un capteur de pression atmosphérique et de température intégrant un convertisseur analogique-numérique (CAN). Ce composant permet de convertir les signaux analogiques du capteur en données numériques, facilitant ainsi leur traitement par un microcontrôleur, essentiel pour des mesures précises dans les applications météorologiques.

2) Le BMP280 propose trois modes de fonctionnement adaptés à différents besoins :

Mode Sleep : Par défaut, ce mode permet de minimiser la consommation d'énergie lorsque des mesures fréquentes ne sont pas nécessaires.

Mode Normal : Le capteur alterne entre des périodes actives et des périodes de veille, idéal pour des mesures régulières et continues, comme dans la surveillance des conditions météorologiques.

Mode Forced : Le capteur effectue une seule mesure, puis retourne en mode Sleep, adapté pour des mesures ponctuelles.

3) Le BMP280 est équipé d’un filtre IIR, permettant de réduire les fluctuations des données de pression, avec un coefficient ajustable pour minimiser les perturbations dues à des facteurs comme le vent.

Choix de l'I2C : J'ai opté pour le bus I2C pour la connexion et la programmation du BMP280. Voici les avantages de ce choix :

Simplicité : L'I2C nécessite seulement deux fils (SDA et SCL) et un fil pour la terre (GND), contrairement à l'SPI qui nécessite plus de fils.

Économie d'énergie : La consommation en mode I2C est de 2,7 µA à 1 Hz, bien plus faible que l'SPI.

Mise en réseau facile : L'I2C permet de connecter jusqu'à 127 dispositifs sur le même bus, avec des adresses uniques pour chaque capteur (0x76 et 0x77 pour le BMP280).

Vitesse de communication : Bien que l'SPI soit plus rapide (jusqu'à 10 MHz), l'I2C avec une vitesse de 3,4 MHz est suffisante pour le BMP280.

Simplicité de programmation : L'I2C est plus simple à gérer dans le code, avec une gestion des transactions de lecture et d'écriture plus intuitive.

Filtrage numérique : Le filtre IIR du BMP280 est plus facilement géré via I2C.

En conclusion, l'I2C est le choix optimal pour ce projet de station météorologique en raison de sa simplicité, de son efficacité énergétique, de sa capacité à gérer plusieurs capteurs et de sa vitesse de communication adaptée.

4) Dans cette partie, je vais me concentrer sur les registres du BMP280 qui seront importants pour mon code. Ces registres permettent de configurer le capteur, récupérer les données et ajuster des paramètres comme le mode de fonctionnement, le taux d'échantillonnage et le filtrage numérique.

Tout d’abord,Les adresses I²C **0x76** et **0x77** du BMP280 sont utilisées respectivement lorsque la broche SDO est connectée à la terre (GND) et à VDDIO (tension d'alimentation), permettant ainsi la communication avec le capteur sur le bus I²C. Ces adresses uniques sont essentielles pour éviter les conflits lors de la connexion de plusieurs capteurs, garantissant un fonctionnement correct dans un système de mesure.

Register 0xF4 *"ctrl_meas"*	Name	Description
Bit 7, 6, 5	osrs_t[2:0]	Controls oversampling of temperature data. See chapter 3.3.2 for details.
Bit 4, 3, 2	osrs_p[2:0]	Controls oversampling of pressure data. See chapter 3.3.1 for details.
Bit 1, 0	mode[1:0]	Controls the power mode of the device. See chapter 3.6 for details.

Ensuite **Registre de Contrôle (0xF4) :** Ce registre configure les options d'acquisition de données, y compris les paramètres d'oversampling pour la pression et la température, ainsi que le mode de fonctionnement (Sleep, Normal, Forced). Cela permet une configuration facile via I²C.

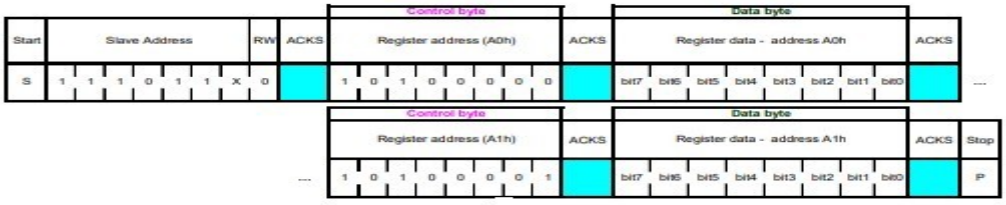
Register 0xF7-0xF9 *"press"*	Name	Description
0xF7	press_msb[7:0]	Contains the MSB part up[19:12] of the raw pressure measurement output data.
0xF8	press_lsb[7:0]	Contains the LSB part up[11:4] of the raw pressure measurement output data.
0xF9 (bit 7, 6, 5, 4)	press_xlsb[3:0]	Contains the XLSB part up[3:0] of the raw pressure measurement output data. Contents depend on temperature resolution, see table 5.

Enfin Les registres **0xF7**, **0xF8**, et **0xF9** du BMP280 contiennent respectivement les 8 bits les plus significatifs (MSB), les 8 bits suivants (LSB), et les 4 bits les moins significatifs (XLSB) de la mesure de pression atmosphérique.

5) Trame de Communication I²C avec le BMP280.

Dans une communication I²C, le **maître (Master)** est le dispositif qui initie et contrôle le flux de données, comme un microcontrôleur (par exemple, un Arduino ou Raspberry Pi) qui envoie des commandes aux capteurs. Le **esclave (Slave)**, comme le BMP280 dans ce cas, répond aux requêtes du maître en envoyant des données de pression et de température en retour, permettant ainsi une interaction fluide entre les dispositifs sur le bus I²C.

-Écriture dans le Registre I²C :



Le maître commence la communication en générant un signal de démarrage(bit start), suivi de l'envoi de l'adresse de l'esclave en mode écriture (RW = '0'). L'esclave répond par un accusé de réception (ACK). Envoi des Données

Le maître envoie ensuite les paires d'adresses de registre et de données, chaque envoi étant confirmé par un ACK de l'esclave. La transaction se termine par un signal d'arrêt(bit de stop), indiquant la fin de l'envoi des données.

Démarrage de la Communication :

- Le maître (ex. Arduino) génère un signal de démarrage.
- **Envoi de l'Adresse de l'Esclave** :
- Le maître envoie l'adresse de l'esclave (0x76 ou 0x77) en mode écriture (RW = '0').
- **Envoi des Données** :
- Le maître envoie l'adresse du registre F4 (0xF4) et la valeur 0x27 pour configurer la fréquence d'échantillonnage.
- **Fin de la Transaction** :
- Le maître envoie un signal d'arrêt pour terminer la communication.

-Lecture dans le Registre I²C :

Envoi de l'Adresse de Registre :

- Le maître envoie l'adresse I²C du BMP280 en mode écriture (RW = '0') et spécifie l'adresse du registre à lire (par exemple, le registre de pression à 0xF7).
- **Passage en Mode Lecture** :
- Le maître génère un signal de reprise (Repeated Start Condition) et adresse l'esclave en mode lecture (RW = '1') pour obtenir les données du registre.

- **Envoi des Données** :
- L'esclave envoie les données demandées (par exemple, 3 octets pour la pression).
- Le maître reçoit chaque octet et envoie un ACK après chaque octet reçu.
- **Fin de la Lecture** :
- Le maître envoie un NACK après le dernier octet pour indiquer la fin de la lecture.
- Le maître envoie un signal d'arrêt pour terminer la communication.

6) Le problème de pression brute trop élevée affichée par le moniteur série provient du fait que le BMP280 fournit une valeur brute qui doit être corrigée. En consultant la datasheet du capteur, j'ai compris qu'il était nécessaire d'appliquer une formule de compensation en utilisant les coefficients internes du capteur.

```
// Returns temperature in DegC, resolution is 0.01 DegC. Output value of "5123" equals 51.23 DegC.
// t_fine carries fine temperature as global value
BMP280_S32_t t_fine;
BMP280_S32_t bmp280_compensate_T_int32(BMP280_S32_t adc_T)
{
    BMP280_S32_t var1, var2, T;
    var1  = ((((adc_T>>3) - ((BMP280_S32_t)dig_T1<<1))) * ((BMP280_S32_t)dig_T2)) >> 11;
    var2  = (((((adc_T>>4) - ((BMP280_S32_t)dig_T1)) * ((adc_T>>4) - ((BMP280_S32_t)dig_T1))) >> 12) *
        ((BMP280_S32_t)dig_T3)) >> 14;
    t_fine = var1 + var2;
    T  = (t_fine * 5 + 128) >> 8;
    return T;
}
""-
// Returns pressure in Pa as unsigned 32 bit integer in Q24.8 format (24 integer bits and 8 fractional bits)
// Output value of "24674867" represents 24674867/256 = 96386.2 Pa = 963.862 hPa
BMP280_U32_t bmp280_compensate_P_int64(BMP280_S32_t adc_P)
{
    BMP280_S64_t var1, var2, p;
    var1 = ((BMP280_S64_t)t_fine) - 128000;
    var2 = var1 * var1 * (BMP280_S64_t)dig_P6;
    var2 = var2 + ((var1*(BMP280_S64_t)dig_P5)<<17);
    var2 = var2 + (((BMP280_S64_t)dig_P4)<<35);
    var1 = ((var1 * var1 * (BMP280_S64_t)dig_P3)>>8) + ((var1 * (BMP280_S64_t)dig_P2)<<12);
    var1 = (((((BMP280_S64_t)1)<<47)+var1))*((BMP280_S64_t)dig_P1)>>33;
    if (var1 == 0)
    {
        return 0; // avoid exception caused by division by zero
    }
    p = 1048576-adc_P;
    p = (((p<<31)-var2)*3125)/var1;
    var1 = (((BMP280_S64_t)dig_P9) * (p>>13) * (p>>13)) >> 25;
    var2 = (((BMP280_S64_t)dig_P8) * p) >> 19;
    p = ((p + var1 + var2) >> 8) + (((BMP280_S64_t)dig_P7)<<4);
    return (BMP280_U32_t)p;
```

Register Address LSB / MSB	Register content	Data type
0x88 / 0x89	dig_T1	unsigned short
0x8A / 0x8B	dig_T2	signed short
0x8C / 0x8D	dig_T3	signed short
0x8E / 0x8F	dig_P1	unsigned short
0x90 / 0x91	dig_P2	signed short
0x92 / 0x93	dig_P3	signed short
0x94 / 0x95	dig_P4	signed short
0x96 / 0x97	dig_P5	signed short
0x98 / 0x99	dig_P6	signed short
0x9A / 0x9B	dig_P7	signed short
0x9C / 0x9D	dig_P8	signed short
0x9E / 0x9F	dig_P9	signed short
0xA0 / 0xA1	reserved	reserved

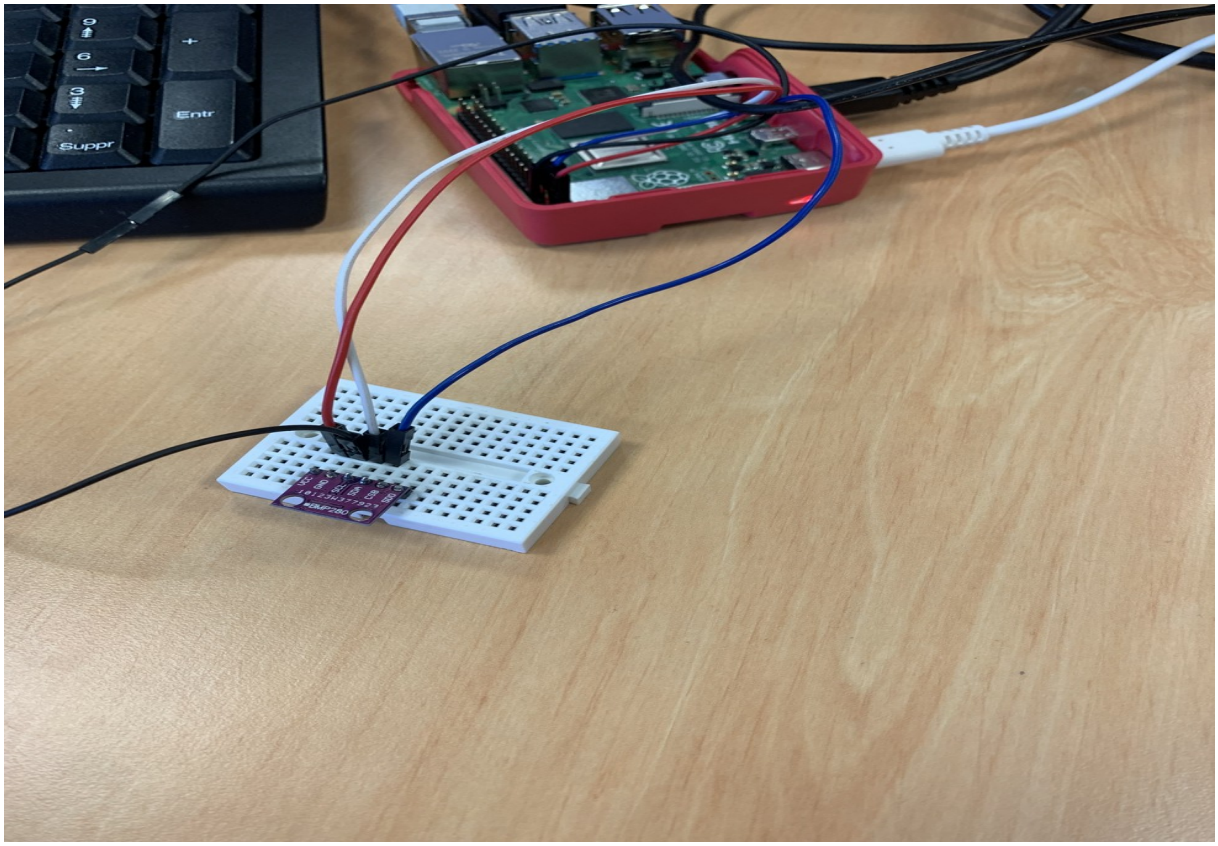
Le capteur BMP280, utilisé pour mesurer la pression atmosphérique, ne fournit pas directement une valeur de pression en hPa, mais plutôt une valeur brute appelée "rawPressure". Cette

valeur doit être corrigée à l'aide des coefficients de calibration internes du capteur.

Selon la datasheet du BMP280, pour obtenir la pression réelle en hPa, il est nécessaire de compenser la valeur brute à l'aide des coefficients de calibration spécifiques au capteur. Ces coefficients sont stockés dans le capteur et doivent être utilisés dans une formule de compensation afin de convertir la pression brute en une valeur exacte.

Câblage:

Dans un premier temps, je vais câbler mon capteur BMP280 à mon Raspberry Pi via l'interface I2C. Le BMP280 utilise deux fils pour la communication : SDA (Serial Data) pour les données et SCL (Serial Clock) pour l'horloge. En plus de ces deux fils, je vais connecter les pins GND (masse) et VCC (alimentation) du capteur. Puisque le BMP280 fonctionne à 3,3V, je vais l'alimenter avec la broche 3,3V du Raspberry Pi.Cela me permet de relier le capteur à un microcontrôleur plus puissant et plus adapté aux besoins de mon projet, conformément aux exigences de mon professeur.



Programmation:

Dans cette deuxième partie, je vais expliquer le programme en détail, en décrivant chaque section du code et son fonctionnement.

1) Inclusion des bibliothèques les bibliothèques nécessaires.

```
#include <iostream>      // Pour afficher les données
#include <fcntl.h>       // Pour manipuler les fichiers (open)
#include <unistd.h>      // Pour les fonctions comme close(), usleep()
#include <sys/ioctl.h>   // Pour les commandes spécifiques aux périphériques (ioctl)
#include <linux/i2c-dev.h> // Définitions pour utiliser I2C sur Linux
#include <cmath>         // Pour les calculs mathématiques (compensation de pression)
#include <cstdint>       // Types de données fixes (int16_t, uint16_t, etc.)
```

2) Définition des constantes.

```cpp
#define BMP280_ADDRESS 0x76
#define REG_CONTROL 0xF4
#define REG_PRESS_MSB 0xF7
#define REG_CALIB 0x88
```

BMP280_ADDRESS : Adresse I²C du capteur (0x76 ou 0x77 selon SDO).
REG_CONTROL : Registre de configuration du capteur.
REG_PRESS_MSB : Registre contenant la pression brute.
REG_CALIB : Adresse de départ des coefficients de calibration.

3) Définition de la structure pour la calibration

```
struct BMP280_CalibData {
    uint16_t dig_T1;
    int16_t dig_T2, dig_T3;
    uint16_t dig_P1;
    int16_t dig_P2, dig_P3, dig_P4, dig_P5, dig_P6, dig_P7, dig_P8, dig_P9;
};
```

Contient les coefficients de calibration stockés en mémoire interne du BMP280. Ces valeurs sont nécessaires pour convertir la pression brute en hPa. Les coefficients sont situés dans la mémoire de 0x88 à 0xA1

4) Lecture de 16 bits depuis un registre I2C

```
int16_t read16(int file, uint8_t reg) {
    uint8_t data[2];
    write(file, &reg, 1);
    read(file, data, 2);
    return (data[1] << 8) | data[0];
}
```

`write(file, &reg, 1);` : Envoie l'adresse du registre.

`read(file, data, 2);` : Lit **2 octets** de données.

`(data[1] << 8) | data[0]` : Reconstitue la valeur en **16 bits**

5) Lecture des coefficients de calibration

```
void readCalibrationData(int file, BMP280_CalibData &calib) {
    calib.dig_T1 = read16(file, 0x88);
    calib.dig_T2 = read16(file, 0x8A);
    calib.dig_T3 = read16(file, 0x8C);
    calib.dig_P1 = read16(file, 0x8E);
    calib.dig_P2 = read16(file, 0x90);
    calib.dig_P3 = read16(file, 0x92);
    calib.dig_P4 = read16(file, 0x94);
    calib.dig_P5 = read16(file, 0x96);
    calib.dig_P6 = read16(file, 0x98);
    calib.dig_P7 = read16(file, 0x9A);
    calib.dig_P8 = read16(file, 0x9C);
    calib.dig_P9 = read16(file, 0x9E);
}
```

Utilise `read16()` pour récupérer les coefficients stockés dans les registres de 0x88 à 0x9E.

Ces valeurs sont indispensables pour la compensation de la pression.

6) Compensation de la pression

```
double compensatePressure(int32_t adc_P, int32_t t_fine, BMP280_CalibData &calib) { ...
```

La pression brute doit être compensée avec les coefficients dig_P1 à dig_P9.
La pression finale est donnée en hPa(datasheet).

7) **<u>Dans cette section du code, nous initialisons la communication avec le capteur BMP280 via le bus I²C.</u>**

```
const char *device = "/dev/i2c-1";
int file = open(device, O_RDWR);
if (file < 0) {
    std::cerr << "❌ Erreur : Impossible d'ouvrir le bus I2C\n";
    return 1;
}

if (ioctl(file, I2C_SLAVE, BMP280_ADDRESS) < 0) {
    std::cerr << "❌ Erreur : Impossible de communiquer avec le BMP280\n";
    close(file);
    return 1;
}
```

`open()` ouvre la communication avec le bus I²C mode écriture/lecture
`ioctl()` définit l'adresse du **BMP280** (0x76 ou 0x77).

<u>8) Dans cette section, nous configurons le capteur BMP280.</u>

```
char config[2] = {REG_CONTROL, 0x57};
if (write(file, config, 2) != 2) {
    std::cerr << "❌ Erreur : Échec de la configuration du BMP280\n";
    close(file);
    return 1;
}
```

Méthode pour configurer Mon capteur à l'aide de la datasheet :
Pour cela j'ai vu sur la dataseet qui permet de configurer le capteur c'est le registre F4.

Register Name	Address	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Reset state
temp_xlsb	0xFC	temp_xlsb<7:4>				0	0	0	0	0x00
temp_lsb	0xFB	temp_lsb<7:0>								0x00
temp_msb	0xFA	temp_msb<7:0>								0x80
press_xlsb	0xF9	press_xlsb<7:4>				0	0	0	0	0x00
press_lsb	0xF8	press_lsb<7:0>								0x00
press_msb	0xF7	press_msb<7:0>								0x80
config	0xF5	t_sb[2:0]			filter[2:0]				spi3w_en[0]	0x00
ctrl_meas	0xF4	osrs_t[2:0]			osrs_p[2:0]			mode[1:0]		0x00
status	0xF3					measuring[0]			im_update[0]	0x00
reset	0xE0	reset[7:0]								0x00
id	0xD0	chip_id[7:0]								0x58
calib25...calib00	0xA1...0x88	calibration data								individual

Registers:	Reserved registers	Calibration data	Control registers	Data registers	Status registers	Revision	Reset
Type:	do not write	read only	read / write	read only	read only	read only	write only

Donc en voyant F4 : on peut apercevoir qu'il nous faut trois autres informations osrs_t osrs_p, mode

Table 22: Register settings osrs_t

osrs_t[2:0]	Temperature oversampling
000	Skipped (output set to 0x80000)
001	oversampling ×1
010	oversampling ×2
011	oversampling ×4
100	oversampling ×8
101, 110, 111	oversampling ×16

osrs_p[2:0]	Pressure oversampling
000	Skipped (output set to 0x80000)
001	oversampling ×1
010	oversampling ×2
011	oversampling ×4
100	oversampling ×8
101, Others	oversampling ×16

mode[1:0]	Mode
00	Sleep mode
01 and 10	Forced mode
11	Normal mode

Étant donné que j'ai choisi le oversmpling x16 pour la pression et x2 pour la température et le mode normal en les mettant dans l'ordre en trouve : Ob 01010111 -> 0x 57

9) Ensuite, une boucle for est utilisée pour itérer sur le nombre de relevés spécifié.

La boucle commence avec `int i = 0;`, où i sert de compteur, et continue tant que i est inférieur à `nombreReleves`. À chaque itération, i est incrémenté de 1 grâce à `i++`. À l'intérieur de la boucle, la fonction `usleep(1000);` est appelée pour introduire une pause de 1000 microsecondes (ou 1 milliseconde), permettant ainsi d'attendre que la mesure soit terminée avant de procéder à la prochaine opération

```
// Variable pour le nombre de relevés
int nombreReleves = 15; // Changez cette valeur pour le nombre de relevés souhaité

for (int i = 0; i < nombreReleves; i++) {
    // Attendre que la mesure soit terminée
    usleep(1000);
```

6)Lecture et affichage de la pression.

```
char reg = REG_PRESS_MSB;
char data[3];

write(file, &reg, 1);
read(file, data, 3);

int32_t rawPressure = ((int32_t)data[0] << 12) | ((int32_t)data[1] << 4)
int32_t t_fine = 10000; // Simulation
double pressure_hPa = compensatePressure(rawPressure, t_fine, calib);

std::cout << "📊 Pression = " << pressure_hPa << " hPa" << std::endl;
```

-Lit les 3 octets de pression (`0xF7` à `0xF9`).
-Reconstitue la valeur brute (20 bits).
-Applique la compensation et affiche la pression en hPa.

D'après la datasheet, les bits de poids forts se situent entre les positions [19 ; 12], les bits de poids faibles entre [11 ; 4], et les bits les moins significatifs entre [3 ; 0]. Les autres bits sont à ignorer.

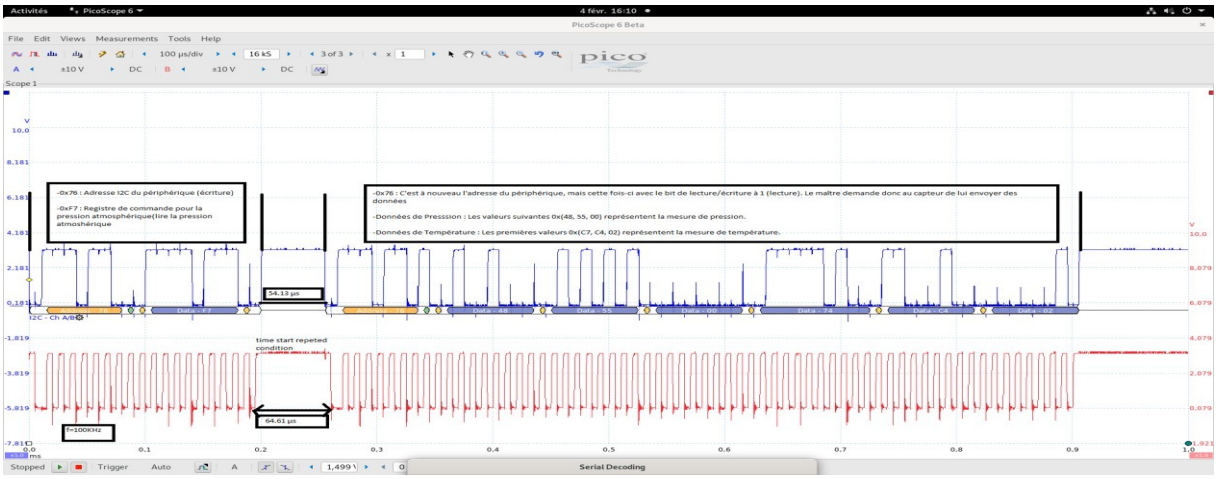
Register 0xF7-0xF9 *"press"*	Name	Description
0xF7	press_msb[7:0]	Contains the MSB part up[19:12] of the raw pressure measurement output data.
0xF8	press_lsb[7:0]	Contains the LSB part up[11:4] of the raw pressure measurement output data.
0xF9 (bit 7, 6, 5, 4)	press_xlsb[3:0]	Contains the XLSB part up[3:0] of the raw pressure measurement output data. Contents depend on temperature resolution, see table 5.

Moniteur Série :

```
Relevé 1 : Pression = 985.109 hPa
Relevé 2 : Pression = 985.08 hPa
Relevé 3 : Pression = 985.061 hPa
Relevé 4 : Pression = 985.062 hPa
Relevé 5 : Pression = 985.086 hPa
Relevé 6 : Pression = 985.089 hPa
Relevé 7 : Pression = 985.086 hPa
Relevé 8 : Pression = 985.155 hPa
Relevé 9 : Pression = 985.125 hPa
Relevé 10 : Pression = 985.097 hPa
Relevé 11 : Pression = 985.144 hPa
Relevé 12 : Pression = 985.072 hPa
Relevé 13 : Pression = 985.107 hPa
Relevé 14 : Pression = 985.086 hPa
Relevé 15 : Pression = 985.072 hPa
```

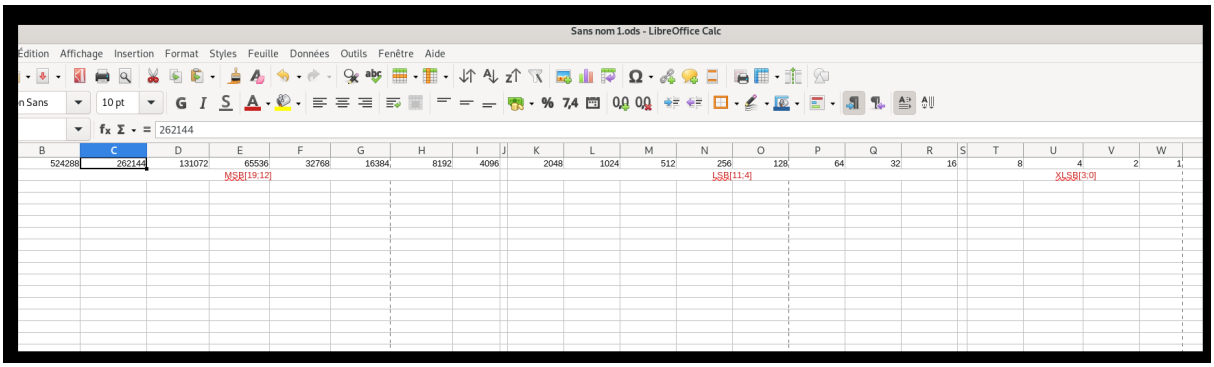
Picoscope :

Dans cette dernière partie, je vais relever les chronogrammes à l'aide d'un PicoScope 6 pour analyser les différentes phases, telles que la transition du maître, l'écriture et la lecture, ainsi que la condition de "Répète Start".



De plus, je vais présenter d'autres relevés pour illustrer le passage du maître du mode écriture au mode lecture. Enfin, j'ai calculé la période afin de déterminer la fréquence de l'horloge et ainsi identifier le mode de l'horloge, qu'il soit en mode standard, fast, etc...

Dans cette étape, je vais calculer la pression à la main à l'aide d'un fichier Excel et vérifier si les résultats correspondent à ceux affichés sur le moniteur série de la Raspberry Pi.



Sur le chronogramme, les données relatives à la pression atmosphérique sont les suivantes : 0x48, 0x55, 0x00.
En binaire, cela donne :

- 0x48 → 0b 0100 1000 (MSB)
- 0x55 → 0b 0101 0101 (LSB)
- 0x00 → 0b 0000 (XLSB)

Les autres bits sont insignifiants. Pour obtenir la valeur de la pression, on doit additionner les valeurs correspondantes aux bits activés :

262144 + 32768 + 1024 + 256 + 64 + 16 = 296 272.

Cependant, cette pression brute ne correspond pas à la pression atmosphérique exacte, car elle n'a pas été compensée en tenant compte des coefficients internes du capteur. Ainsi, les valeurs mesurées ne reflètent pas la pression réelle en raison de l'absence de cette compensation.

Conclusion du BMP280 :

Le BMP280 est un capteur de pression atmosphérique précis et polyvalent, capable de mesurer la pression et la température avec une faible consommation d'énergie. Il peut communiquer avec un microcontrôleur via **I²C** ou **SPI**, ce qui le rend compatible avec de nombreuses cartes comme **Arduino, ESP8266 et Raspberry Pi**.

Capteur anémomètre:

Dans le cadre de mon projet de ballon anti-grêle, l'utilisation de l'anémomètre **Peet Bros** est essentielle pour surveiller la vitesse et la direction du vent en temps réel. En météorologie, le vent joue un rôle crucial dans la formation et la dynamique des orages, notamment ceux susceptibles de produire de la grêle. Plus le vent est fort, plus il peut favoriser les courants ascendants puissants au sein des nuages convectifs, ce qui augmente les chances de formation et de maintien des grêlons avant qu'ils ne tombent au sol.

Grâce à la précision et à la robustesse de l'anémomètre **Peet Bros**, il sera possible de collecter des données fiables sur la vitesse du vent à différentes altitudes. Ces informations permettront d'anticiper les conditions favorables aux chutes de grêle et d'optimiser le déclenchement des dispositifs de protection embarqués sur le ballon. L'intégration de cet anémomètre au système global de surveillance et d'alerte météo améliore ainsi la réactivité et l'efficacité du projet dans la lutte contre les épisodes de grêle.

Enfin je vais consulter la datasheet de ce capteur afin de vérifier s'il répond parfaitement aux exigences du projet et d'analyser ses spécificités techniques. Voici le lien vers sa documentation :
https://www.peetbros.com/shop/custom.aspx?recid=14

Avant de faire mon choix définitif, j'ai d'abord analysé la datasheet, qui répertorie quatre modèles d'anémomètres. Après lecture, j'ai identifié que **l'Ultimeter Pro** correspondait le mieux aux exigences du projet. De plus, en classe, j'ai pu comparer les modèles disponibles et confirmer qu'il s'agissait bien de celui utilisé par le professeur.

DATASHEET

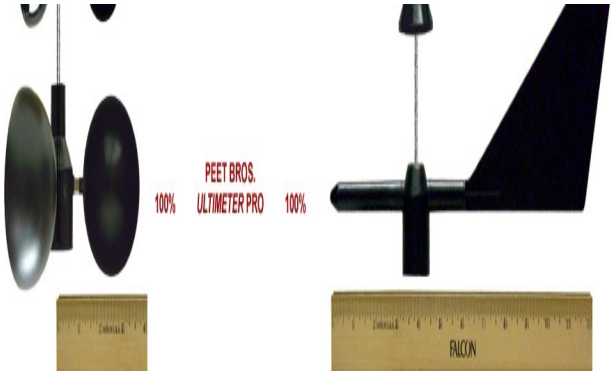
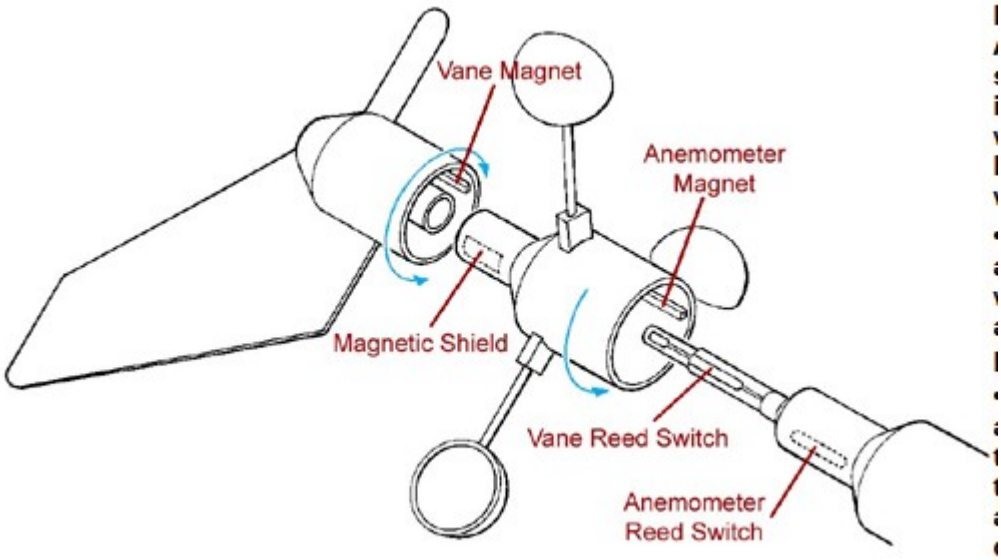


Photo Anémomètre Classe

Dans cette section, je vais expliquer la composition de cet anémomètre-girouette. Il est constitué de plusieurs

éléments clés, notamment une girouette, un interrupteur Reed et un système basé sur des aimants et des coupelles. Je détaillerai le principe de fonctionnement, en expliquant comment les aimants interagissent avec les interrupteurs Reed pour mesurer la vitesse et la direction du vent. Enfin, en m'appuyant sur la datasheet, j'indiquerai dans quelles conditions l'interrupteur Reed est ouvert ou fermé.



Ce schéma montre le fonctionnement de l'anémomètre Ultimeter Pro de Peet Bros. Il utilise un système de magnétisme et d'interrupteurs Reed pour mesurer la vitesse et la direction du vent moi je vais me concentrer dans ce schéma sur la vitesse du vent l'anémomètre en lui-même et non pas la girouette

Fonctionnement :

Mesure de la vitesse du vent : l'anémomètre possède un aimant (Anemometer Magnet) fixé à l'axe de rotation des coupelles. À chaque tour complet des coupelles, cet aimant passe devant un interrupteur Reed (Anemometer Reed Switch), qui ferme brièvement un circuit électrique. La fréquence des impulsions électriques permet de calculer la vitesse du vent.

Dans la seconde partie, il est essentiel de déterminer si le relais Reed est normalement ouvert ou fermé en s'appuyant sur les informations fournies dans la datasheet.

D'après la datasheet, le fonctionnement des reed switches (interrupteurs à lames souples) dans l'anémomètre Peet Bros ULTIMETER est le suivant :

-Reed switch de l'anémomètre (vitesse du vent) :
Normalement ouvert, il se ferme brièvement à chaque passage de l'aimant attaché aux coupelles, puis se rouvre une fois l'aimant passé.
Cette fermeture périodique permet de compter le nombre de tours effectués par l'anémomètre et ainsi de calculer la vitesse du vent.

Dans cette dernière section, nous allons examiner en détail les spécificités techniques de cet anémomètre en nous appuyant sur les informations fournies par une autre datasheet : https://www.peetbros.com/PROAWVFLYER.pdf
Je vais construire un tableau pour mieux visualiser les spécificités techniques de anémomètre :

Caractéristique	Spécifications	Pourquoi c'est utile pour le projet ?
Type	Anémomètre à 3 coupelles, numérique, activation magnétique	Permet de mesurer précisément la vitesse du vent, un facteur clé dans la formation de la grêle
Matériaux	Polycarbonate résistant aux UV, Acier inoxydable	Résiste aux conditions extérieures, parfait pour une utilisation prolongée en ballon météo
Plage de température	-48°C à +65°C	Fonctionne dans des conditions météo extrêmes, idéal pour surveiller les tempêtes
Taille des coupelles	6,6 cm de diamètre	Plus grandes coupelles = meilleure sensibilité au vent, donc des mesures plus précises
Résolution	0.1 km/h	Détecte les plus petits changements de vitesse du vent, utile pour anticiper un orage
Plage de mesure	0.3 à 257 km/h	Capable de mesurer aussi bien de faibles

		brises que des vents violents liés aux orages
Seuil de détection	2.4 km/h	Commence à mesurer le vent dès qu'il dépasse 2.4 km/h, ce qui permet un suivi continu
Fréquence de mise à jour	Toutes les 0.5 secondes plus rapide si vent>5mph	Permet de suivre les changements brusques de vent, essentiels pour prévoir la grêle
Précision	± 3.2 km/h ou ±5%	Mesures fiables pour analyser le lien entre vent fort et risque de grêle
Connexion	Câble 4 fils de 12m	Facile à connecter aux autres capteurs du ballon

Facteur de conversion :

Étape 1 : Circonférence des coupelles

La circonférence des coupelles est la distance parcourue par les coupelles en un tour complet. Elle est donnée par la formule : Circonférence=π×Diamètre

Avec un diamètre de 6.6 cm (donné dans la datasheet) car 2,6 pouce :

Circonférence=π×6.6cm≈20.7cm(ou0.207m)

Étape 2 : Calcul théorique du facteur de conversion

-Distance par impulsion : Chaque impulsion correspond à une rotation complète des coupelles, soit 0.207m. Vitesse(mph)=0.207m/s×(1609.34m3600s)≈0.463mph. Problème : Ce facteur de 0.463 mph par impulsion ne couvre pas la plage de mesure de l'anémomètre (1.5 à 160 mph).

Étape 3 : Ajustement du facteur à 2.5 mph

Seuil de démarrage : L'anémomètre commence à générer des impulsions à 1.5 mph. Tests d'étalonnage de Peet Bros : Le fabricant a ajusté le facteur de conversion pour correspondre à des tests en conditions réelles.Facteur

retenu : 2.5 mph par impulsion, permettant une mesure précise dans la plage de 1.5 à 160 mph.

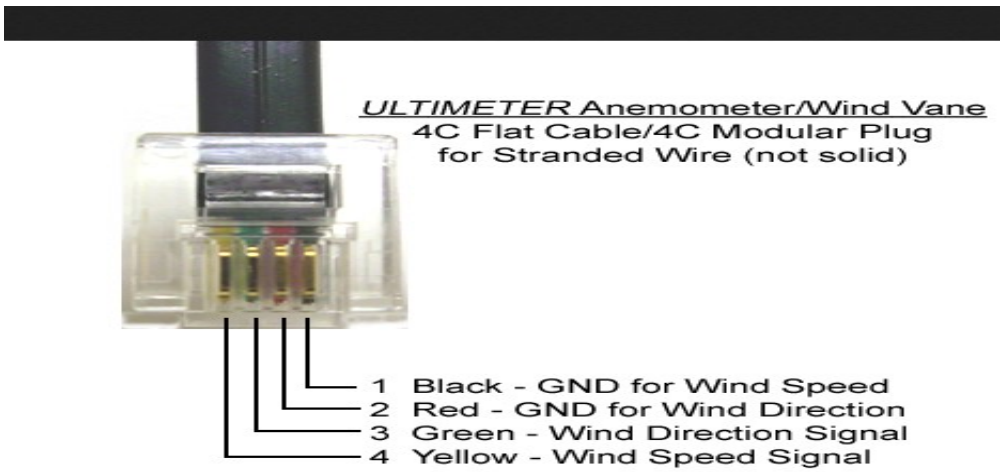
Le facteur de conversion de 2.5 mph par impulsion est une valeur ajustée basée sur les tests d'étalonnage de Peet Bros. Il permet une mesure précise de la vitesse du vent dans la plage de 1.5 à 160 mph, conformément aux spécifications techniques de l'anémomètre.

Étude matérielle :

L'anémomètre est équipé de 4 coupelles de vent de 2,6 pouces de diamètre. Il peut être utilisé avec des câbles de plus de 250 pieds et est fourni avec un câble à 4 conducteurs de 40 pieds.

Câblage:

Tableau du câblage de l'anémomètre Peet Bros en s'aidant de l'image d'un cablage trouvé sur la datasheet :
https://www.peetbros.com/images/AWV4C.png



Fil de l’anémomètre	Fonction	Connexion Raspberry Pi
Noir	Masse(GND) vitesse	GND (Pin 6)
Rouge	Masse(GND) direction	(Non connecté ici)
Jaune	Signal vitesse du vent	GPIO 17 (Pin 11)
Vert	Signal direction du vent	(Non connecté ici)
Résistance 10kΩ	Pull-up pour tirer le siganl niveau Haut	Entre GPIO 17 et 3.3V (Pin 1)

Adaptation du câblage avec une platine pédagogique RJ45 :

Pour connecter l’anémomètre Peet Bros ULTIMETER PRO à mon système, j’ai utilisé une platine pédagogique RJ45 permettant d'insérer le connecteur RJ11 de l’anémomètre.

- Différence entre RJ45 et RJ11 :

- Le RJ45 possède 8 broches, alors que le RJ11 en possède 4.
- Étant donné que je veux uniquement mesurer la vitesse du vent, seuls deux fils sont nécessaires.

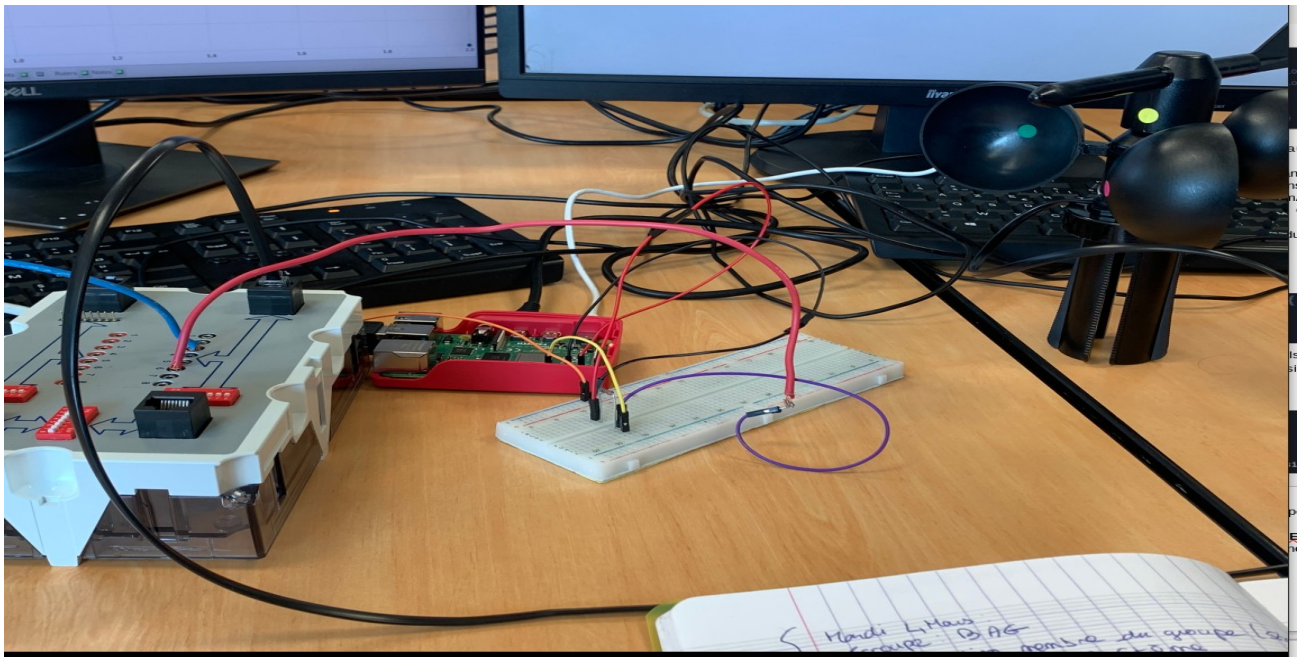
- Détermination des bonnes broches :

- J’ai effectué des tests manuels pour identifier les bonnes connexions.
- Les broches 1-2 et 7-8 ne sont pas utilisées.
- Après plusieurs essais, j’ai déterminé que :
- Les broches 3 et 6 servent à la vitesse du vent.
- Les broches 4 et 5 correspondent à la direction du vent

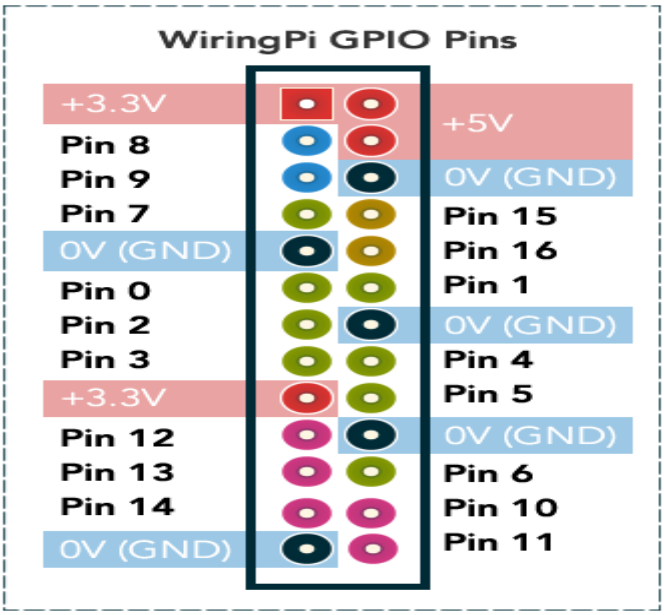
Problème des fils standards :

- Les pins standard ne rentraient pas car ils étaient trop gros.
- J’ai pu récupérer des fils plus adaptés grâce à la classe d'électrotechnique.
- J’ai soudé deux fils mâle-mâle pour créer une connexion stable.

- Les fils ont ensuite été reliés à une breadboard, puis on utilisant des fils mâle-femelle reliées directement au Raspberry Pi.



Remarque :



Dans la bibliothèque WiringPi, le GPIO 17 du Raspberry Pi correspond au pin 0 dans le code

Explication de WiringPi :

WiringPi est une bibliothèque permettant de contrôler facilement les GPIO du Raspberry Pi en C/C++. Elle utilise une numérotation propre différente de la numérotation physique (broche) et BCM (Broadcom). Ainsi, il est essentiel de bien identifier la correspondance entre les numéros de broches et ceux utilisés dans WiringPi pour éviter toute erreur.

En comptant, nous constatons que le GPIO 17 (broche physique 11) du Raspberry Pi correspond au pin 0 dans la bibliothèque WiringPi

Programmation:

1 Inclusion des bibliothèques nécessaires :

```
#include <wiringPi.h>
#include <iostream>
```

-`iostream` : Sert à gérer les entrées/sorties, notamment avec `cout` et `cin`.
-`wiringPi.h` : Permet d'interagir avec les broches GPIO de la Raspberry Pi.

Initialisation des constantes et variables

```
#define REED_PIN 0
const double FACTEUR_MPH = 2.5;      // Facteur d'étalonnage en mph
const double FACTEUR_KMH = 4.0225;   // Facteur d'étalonnage en km/h

volatile int compteur_impulsions = 0;
int nbre_releves = 15;       // Nombre de relevés fixé à 15
```

-**REED_PIN** : Le pin GPIO du Raspberry Pi auquel l'anémomètre est connecté.
-FACTEUR_MPH et **FACTEUR_KMH** : Ces constantes sont des facteurs de conversion pour transformer le nombre d'impulsions en vitesse du vent en miles par heure (mph) et en kilomètres par heure (km/h).
-compteur_impulsions : Une variable volatile qui compte le nombre d'impulsions détectées par l'anémomètre.
-nbre_releves : Le nombre de relevés de vitesse du vent que le programme doit effectuer.

3 Fonction d'interruption

```
void impulsion_detectée() {
    compteur_impulsions++;
}
```

-Cette fonction est appelée chaque fois qu'une impulsion est détectée sur le pin **REED_PIN**. Elle incrémente le compteur d'impulsions.

4 Configuration du GPIO et de l'interruption

```
int main() {
    wiringPiSetup();
    pinMode(REED_PIN, INPUT);
    pullUpDnControl(REED_PIN, PUD_UP);
    wiringPiISR(REED_PIN, INT_EDGE_FALLING, &impulsion_detectée);
}
```

-**wiringPiSetup()** : Initialise la bibliothèque WiringPi pour utiliser les GPIO du Raspberry Pi.
-pinMode(REED_PIN, INPUT) : Configure le pin **REED_PIN** en mode entrée.
-pullUpDnControl(REED_PIN, PUD_UP) : Active une résistance de pull-up interne pour stabiliser le signal.
-wiringPiISR(REED_PIN, INT_EDGE_FALLING, &impulsion_detectée) : Configure une interruption sur le front descendant du signal (lorsque l'impulsion est détectée) et associe cette interruption à la fonction impulsion_detectée.

Boucle principale de mesure

```
for (int i = 0; i < nbre_releves; ++i) {
    delay(1000);  // Attendre 1 seconde

    // Calculer la vitesse du vent
    double vitesse_vent_mph = compteur_impulsions * FACTEUR_MPH;
    double vitesse_vent_kmh = compteur_impulsions * FACTEUR_KMH;

    // Afficher la vitesse du vent et le numéro de relevé
    std::cout << "Vent : " << vitesse_vent_mph << " mph | "
              << vitesse_vent_kmh << " km/h" << std::endl;
    std::cout << "Relevé numéro : " << (i + 1) << std::endl;

    compteur_impulsions = 0;  // Réinitialiser le compteur d'impulsions
}
```

-**delay(1000)** : Attend 1 seconde entre chaque relevé pour permettre à l'anémomètre de générer des impulsions.
-**vitesse_vent_mph** et **vitesse_vent_kmh** : Calculent la vitesse du vent en mph et en km/h en multipliant le nombre d'impulsions par les facteurs de conversion respectifs.
-std::cout : Affiche la vitesse du vent et le numéro du relevé.
-compteur_impulsions = 0 : Réinitialise le compteur d'impulsions pour le prochain relevé.

Fin du programme

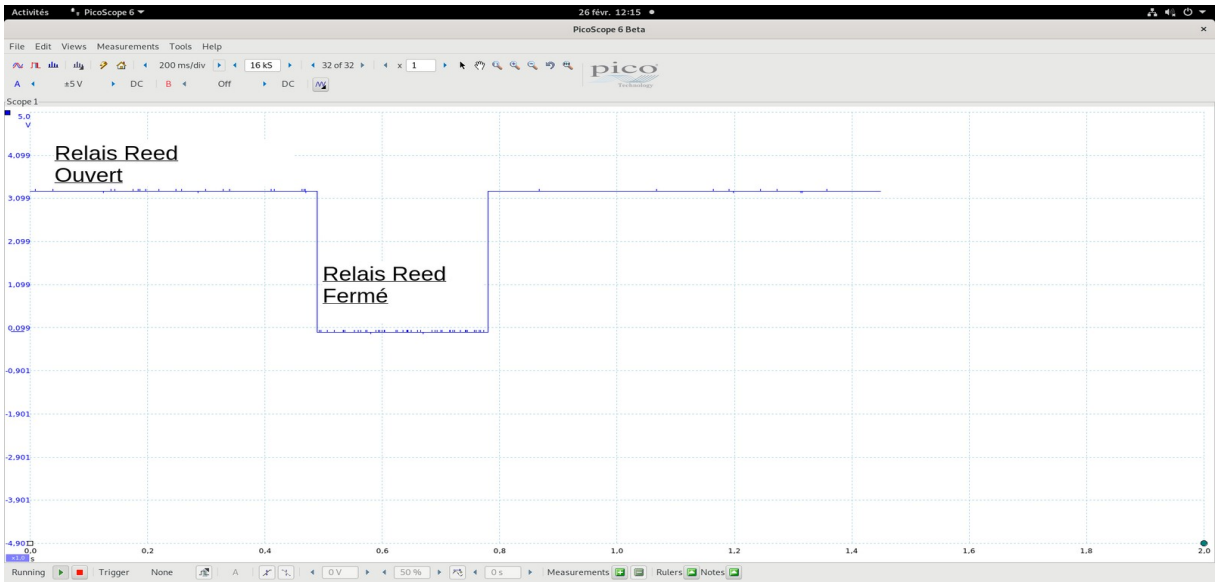
```
std::cout << "Nombre de relevés terminé." << std::endl;

return 0;
```

Moniteur Série :

```
adam@raspberrypi: ~
Fichier  Édition  Onglets  Aide
  * Supports full  user-level GPIO access via memory.
  * Supports basic user-level GPIO access via /dev/gpiomem.
  * Supports basic user-level GPIO access via /dev/gpiochip (slow).
adam@raspberrypi:~ $ nano anemometre.cpp
adam@raspberrypi:~ $ g++ -o anemometre anemometre.cpp -lwiringPi
adam@raspberrypi:~ $ sudo ./anemometre
Combien de relevés souhaitez-vous ? 16
Relevé 1 - Vitesse du vent : 0 mph (0 km/h)
Relevé 2 - Vitesse du vent : 0 mph (0 km/h)
Relevé 3 - Vitesse du vent : 0.5 mph (0.8045 km/h)
Relevé 4 - Vitesse du vent : 4.5 mph (7.2405 km/h)
Relevé 5 - Vitesse du vent : 7.5 mph (12.0675 km/h)
Relevé 6 - Vitesse du vent : 7.5 mph (12.0675 km/h)
Relevé 7 - Vitesse du vent : 7.5 mph (12.0675 km/h)
Relevé 8 - Vitesse du vent : 7 mph (11.263 km/h)
Relevé 9 - Vitesse du vent : 4.5 mph (7.2405 km/h)
Relevé 10 - Vitesse du vent : 9.5 mph (15.2855 km/h)
Relevé 11 - Vitesse du vent : 10 mph (16.09 km/h)
Relevé 12 - Vitesse du vent : 8 mph (12.872 km/h)
Relevé 13 - Vitesse du vent : 12 mph (19.308 km/h)
Relevé 14 - Vitesse du vent : 11.5 mph (18.5035 km/h)
Relevé 15 - Vitesse du vent : 23.5 mph (37.8115 km/h)
Relevé 16 - Vitesse du vent : 4.5 mph (7.2405 km/h)
```

Picoscope:

Je vais effectuer des relevés avec le PicoScope sur l'anémomètre Peet Bros pour observer les impulsions générées par le relais Reed. Je dois obtenir un signal carré où le relais est **ouvert à 3.3V** et **fermé à 0V** à chaque passage de l'aimant.



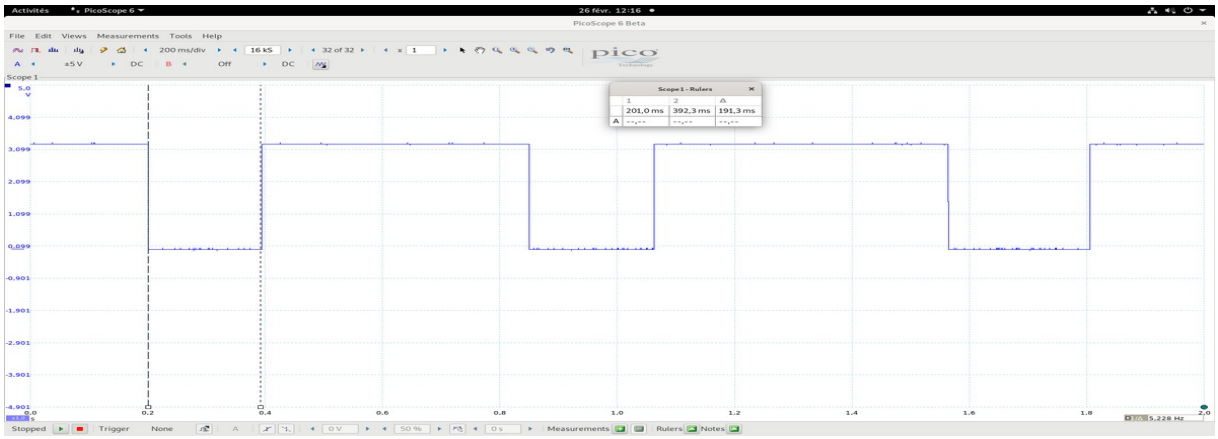
De plus J’ai mesuré le temps de fermeture du relais entre le début et la fin de l’impulsion pour analyser la durée exacte du contact.

La relation entre la vitesse du vent et la durée de fermeture du relais Reed est inversement proportionnelle.

Explication :

- **Vitesse du vent élevée** : Les coupelles tournent plus vite.
- **Passage rapide de l'aimant** : L'aimant passe plus rapidement devant le relais Reed.
- **Durée de fermeture réduite** : La durée pendant laquelle le relais reste fermé diminue à mesure que la vitesse du vent augmente.

Cela confirme que la durée de fermeture du relais Reed diminue lorsque la vitesse du vent augmente, établissant ainsi une relation inversement proportionnelle.



Conclusion Anémomètre :

L’anémomètre Peet Bros ULTIMETER PRO est un capteur précis et fiable pour la mesure du vent, basé sur un relais Reed activé magnétiquement. L’analyse du signal a montré une relation inversement proportionnelle entre la vitesse du vent et la durée de fermeture du relais, confirmant son bon fonctionnement.

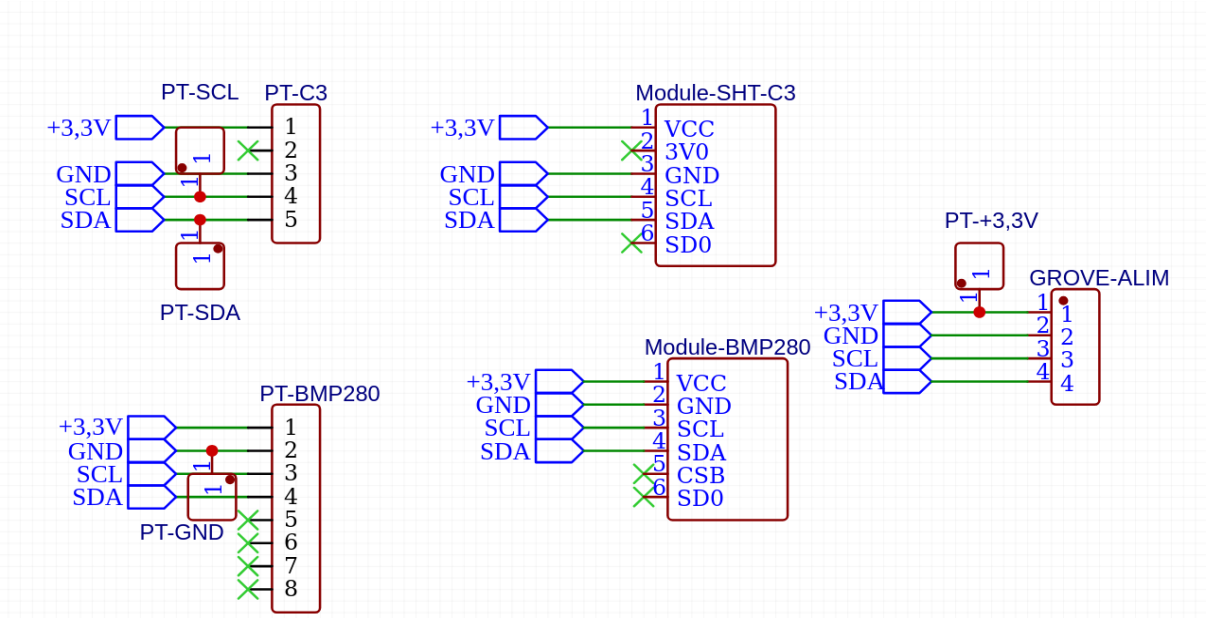
Phase 2 : Concevoir une carte PCB et développer un programme,modélisation 3D.

Carte PCB :

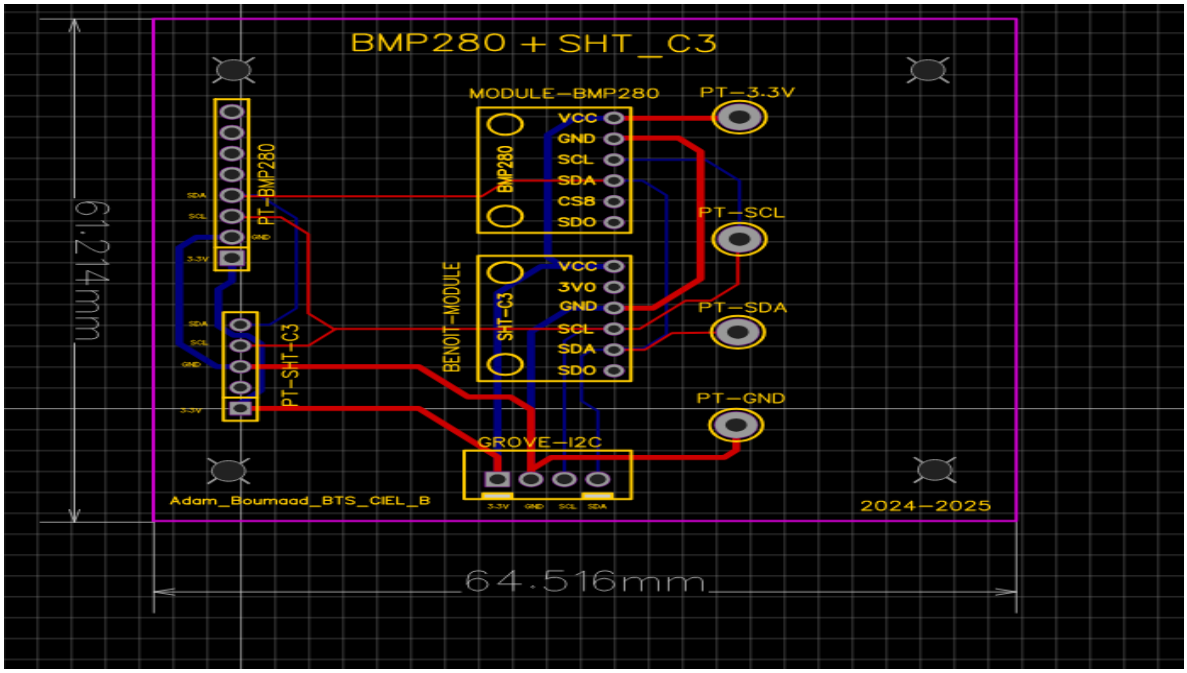
Pour concevoir ma carte PCB, j'intégrerai deux capteurs : le BMP280 (pression atmosphérique) et le SHT-C3 (température et humidité). L'utilisation d'une PCB garantit fiabilité, robustesse et réduit les risques de déconnexion ou de fausses mesures en extérieur. Elle permet aussi une meilleure optimisation de l'espace, essentielle pour son intégration sur la station vigne. La conception se fera sur EasyEDA. Je dois d'abord sélectionner les composants adaptés. Comme mes capteurs utilisent le bus I2C, je privilégie des modules pour simplifier l'intégration et assurer la compatibilité. J'ajouterai un connecteur Grove comme bornier d'alimentation, un seul suffisant grâce au bus I2C pouvant connecter jusqu'à 127 capteurs, réduisant ainsi le câblage et les erreurs. Des points de test seront aussi prévus pour vérifier les signaux et éviter d'éventuels problèmes. Malheureusement, il n'existe pas de module SHT-C3 prêt à l'emploi sur EasyEDA. Donc je vais dupliquer le module bmp280 et en se basant sur les pins des modules sht-c3 je change celui du bmp280 et à la fin je vais souder mon propre SHT-3

Nom du composant	Référence EasyEDA	**Fonction**
Capteur Température & Humidité	GY-BMP280	Mesure la température et l'humidité
Capteur Pression Atmosphérique	GY-BMP280	Mesure la pression atmosphérique
Connecteur Grove	KH-XH-4A-Z.	Permet une connexion simplifiée de l'alimentation
Point Test (SHTC3)	Header-Female-2.54_1x5	Facilite le test et le dépannage du SHTC3
Point Test (BMP280)	Header-Female-2.54_1x8	Facilite le test et le dépannage du BMP280
Points Test VCC, SDA, SCL	5011_C9900013761	Permet de vérifier les signaux d'alimentation et de communication

Deuxièment, Une fois les composants sélectionnés, l'étape suivante consiste à concevoir le schéma structurel sur EasyEDA. Celui-ci permettra de définir les connexions entre les différents éléments et d'assurer leur bon fonctionnement sur la PCB.

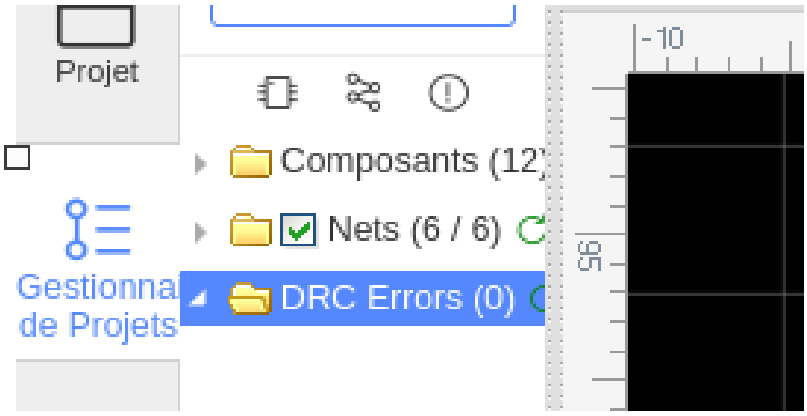


De plus , Si le schéma est correct, on peut le convertir en PCB en suivant les normes de placement et de routage. Il faut éviter les angles droits dans les pistes pour prévenir les pertes de courant, échauffements et courts-circuits, et préférer des angles de 45° ou des courbes douces. Pour **VCC**, une largeur de **0,6 mm** est idéale, tandis que pour **GND**, la largeur dépend des besoins, mais pour les **condos de découplage** ou **pull-up**, une largeur de **0,3 mm** est suffisante. Les signaux **I2C** nécessitent des pistes de **0,254 mm** pour éviter les interférences. Ces normes sont essentielles pour assurer la fiabilité et éviter des problèmes électriques ou thermiques.

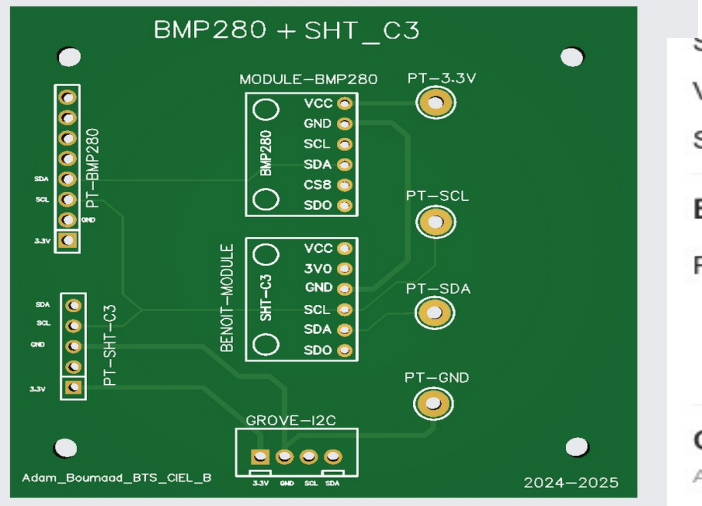


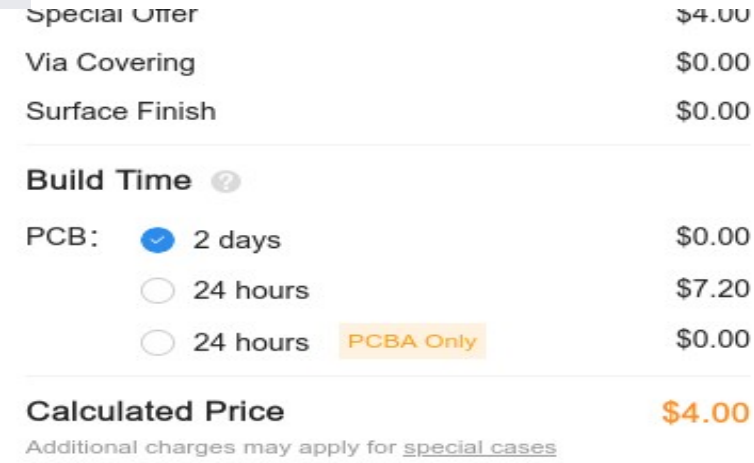
Ensuite, j'ai fait check DRC qui est un outil qui permet de vérifier automatiquement que le design du PCB respecte toutes les règles de

conception définies, telles que les espacements entre les pistes, la largeur des pistes.



Enfin voici le rendu en 3D.





<u>Conclusion:</u> respecter les normes de routage, comme éviter les angles droits et ajuster la largeur des pistes pour VCC, GND, et I2C, assure la fiabilité et la performance du PCB. Le prix pour ces deux PCB sera de 4$.

Programme Seuil d'alerte :

```
#include <iostream>  // Inclusion de la bibliothèque pour l'affichage et la saisie

using namespace std; // Permet d'éviter d'écrire "std::" devant cout et cin

int main() {
    // Déclaration des variables pour stocker les valeurs des capteurs
    float temperature, humidite, pression, vent;

    // 🔴 Simulation : Demande à l'utilisateur d'entrer les valeurs des capteurs
    cout << "Température (°C) : ";
    cin >> temperature; // Lecture de la température saisie par l'utilisateur

    cout << "Humidité (%) : ";
    cin >> humidite; // Lecture du taux d'humidité
```

```
cout << "Pression (hPa) : ";
cin >> pression; // Lecture de la pression atmosphérique

cout << "Vitesse du vent (km/h) : ";
cin >> vent; // Lecture de la vitesse du vent

// 🔴 Définition des seuils de risque de grêle
// Conditions pour un RISQUE MODÉRÉ (niveau 2)
bool risque_moderé = (temperature >= 20 && temperature <= 25) &&  // Température entre 20
                     (humidite >= 60 && humidite <= 70) &&        // Humidité entre 60% e
                     (pression >= 995 && pression <= 1000) &&    // Pression entre 995 hP
                     (vent >= 20 && vent <= 30);                 // Vent entre 20 km/h et

// Conditions pour un RISQUE ÉLEVÉ (niveau 3)
bool risque_eleve = (temperature > 25) &&  // Température supérieure à 25°C
```

```
bool risque_eleve = (temperature > 25) &&  // Température supérieure à 25 °
                    (humidite > 70) &&     // Humidité supérieure à 70%
                    (pression < 995) &&    // Pression inférieure à 995 hPa
                    (vent > 30);           // Vent supérieur à 30 km/h

// 🔴 Vérification des risques et affichage du message correspondant
if (risque_eleve) {
    cout << "⚠️ ALERTE GRELE NIVEAU 3 : RISQUE ÉLEVÉ !" << endl;
}
else if (risque_moderé) {
    cout << "⚠️ ALERTE GRELE NIVEAU 2 : RISQUE MODÉRÉ !" << endl;
}
else {
    cout << "✅ Pas de risque de grêle détecté." << endl;
}

return 0; // Fin du programme
```

<u>Conclusion :</u> Ce programme permet d'envoyer une alerte de niveaux 2 et 3 via le gsm au chef de culture.

Modélisation 3D:

Conclusion :